PROJECT TITLE

GPU-Accelerated Image Compression with Content-Aware Quality Mapping

A Project Component for

Parallel and Distributed Computing (UCS645)

Submitted by

S. No.	Name	Roll No.
1	Gaurang Garg	102303134
2	Aditya Chauhan	102303130
3	Saksham Mittal	102303135

Under the guidance of

Dr. Saif Nalband

Assistant Professor, DCSE

Department of Computer Science and Engineering

Thapar Institute of Engineering and Technology

(Deemed to be University)

Patiala - 147004

May 2026

Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 1

2 Literature Review **3**

3 Research Gaps **5**

4 Problem Formulation **6**

5 Objectives **7**

6 Methodology: Image Compression Pipeline **8**

6.1 Overview of the Compression Approach 8

6.2 Block-Wise Image Decomposition 8

6.3 Forward Discrete Cosine Transform (Forward DCT) 9

6.4 Quantization: Lossy Compression Step 9

6.4.1 Quantization Table Design 10

6.4.2 Quality-Based Scaling 10

6.4.3 Per-Block Coefficient Scaling (Saliency-Aware) 11

6.5 Dequantization and Inverse DCT 11

6.6 Pixel Reconstruction and Clamping 12

6.7 RGB Image Assembly and JPEG Encoding 12

6.8 Quality Assessment Metrics 13

6.8.1 Mean Squared Error (MSE) 13

6.8.2 Peak Signal-to-Noise Ratio (PSNR) 13

6.9 CPU and GPU Implementation Parity 14

6.10 Summary of the Pipeline 14

7 Experimental Setup **16**

8 Results and Discussion **17**

Conclusion **19**

1 . Introduction

1.1 Background

Image compression is an essential requirement in modern graphics, multimedia systems, and cloud-based storage environments. As image resolution and data volume increase, the cost of storing and transmitting visual content grows correspondingly. This project addresses that challenge by implementing a GPU-accelerated image compression pipeline in C++ and CUDA. The system follows a JPEG-like transform workflow based on 8x8 blocks, where each color channel is processed through forward transform, quantization, inverse transform, and reconstruction.

The project is designed as both a functional image compression tool and a benchmarking platform. The GPU path accelerates the most expensive stages of the pipeline, while the CPU reference path remains available for validation, comparison, and debugging. In addition, the implementation supports content-aware quality mapping so that perceptually important regions can retain more detail while less important regions are compressed more aggressively.

1.2 Introduction to Problem Statement

The main problem addressed by this project is how to compress images efficiently while preserving visual quality and making the implementation measurable across CPU and GPU execution paths. Conventional compression tools often hide internal stages, which makes it difficult to understand where time is spent and how quality changes under different parameter settings. In contrast, this project exposes the full processing chain: decoding, planar channel conversion, block-wise transform processing, optional saliency-based weighting, and final JPEG encoding.

The implementation is intended as a practical learning and benchmarking system for parallel and distributed computing. It enables evaluation of CUDA-based parallelization, saliency-driven quality control, and CPU versus GPU behavior on the same image buffers. The resulting tool is useful for studying the performance and quality trade-offs of block-based compression on modern hardware.

2 . Literature Review

Table 2.1 summarizes the main ideas that informed the design of this project.

Table 2.1: Summary of literature review

Ref.	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[1]	JPEG still image compression, transform coding	8x8 DCT, quantization, entropy coding	Establishes the classical baseline for image compression and motivates the block-transform design used in this project	Standard JPEG is not optimized for GPU execution and does not expose intermediate timing or quality-control stages
[2]	GPU parallelism, CUDA programming, throughput-oriented design	SIMT execution, shared memory, constant memory	Provides the architectural foundation for accelerating the block-wise compression pipeline on the GPU	Requires careful memory management and kernel design to avoid divergence and synchronization overhead

Continued on next page

Table 2.1 – continued from previous page

Ref.	Concepts Highlighted	Technique(s) Used	Significance / Proposal	Limitations
[4]	Perceptual quality and saliency-aware processing	Edge detection, local contrast, block importance mapping	Supports content-aware quality mapping so important image regions retain more detail	Saliency-based methods can be image-dependent and may need tuning for different scenes

The literature suggests that classical transform coding is effective for compression, but it also reveals a gap between theoretical compression pipelines and efficient GPU implementations. Many systems also treat all regions of an image uniformly, even though human perception is more sensitive to edges, textures, and detailed areas. This project combines these ideas by using a GPU-friendly block compression pipeline together with optional saliency-aware scaling.

3 . Research Gaps

The major gaps identified for this project are as follows:

1. Most conventional compression demonstrations focus on algorithmic correctness but do not compare CPU and GPU execution in a structured way.
2. Many implementations treat image quality as a global parameter only, without adapting compression strength to local visual importance.
3. Existing examples often hide internal stages, making it difficult to measure saliency, transfer, compute, and encoding time separately.
4. A practical project for parallel and distributed computing should expose both performance behavior and quality behavior, which is missing in many simple codec examples.

These gaps motivate a design that is modular, measurable, and easy to benchmark on sample images with and without saliency-based quality mapping.

4 . Problem Formulation

The objective of this project is to design and implement a GPU-accelerated image compression system that can reconstruct JPEG-like output from RGB input images while retaining a CPU reference path for validation. The system must process images in 8x8 blocks, apply a transform-domain compression step, and write the final output as JPEG using a standard library encoder. At the same time, the implementation should support optional content-aware quality mapping so that visually important regions receive stronger preservation than low-importance regions.

From an engineering perspective, the challenge is not only to compress the image but also to organize the pipeline so that the performance of each stage can be measured independently. The project therefore separates decoding, planar conversion, saliency computation, GPU transfer, kernel execution, CPU comparison, and final JPEG writing. This separation allows the developer to study the behavior of each component and understand how the GPU improves throughput relative to the CPU baseline.

From a quality perspective, the challenge is to preserve visually important regions without significantly increasing complexity. A global quality factor is simple to control but does not adapt to image content. For this reason, the project introduces a quality map based on saliency, local contrast, and edge strength. The result is a compression system that can be tuned for either speed, quality, or a balance of both, depending on the selected configuration.

5 . Objectives

- To implement a GPU-based image compression pipeline using CUDA and 8x8 block processing.
- To maintain a CPU reference implementation for validation and performance comparison.
- To support global compression control through a quality parameter.
- To add optional content-aware quality mapping based on saliency and local contrast.
- To measure saliency, upload, compute, download, encode, and total timings separately.
- To generate JPEG output using a standard libjpeg-based writer.
- To support compare mode so CPU and GPU outputs can be evaluated on the same input image.

6 . Methodology: Image Compression Pipeline

6.1 Overview of the Compression Approach

This project implements a JPEG-like transform-based image compression pipeline that operates on 8×8 pixel blocks. Unlike traditional JPEG encoders, which emit quantized DCT coefficients directly to a bitstream with entropy coding, this implementation reconstructs full RGB pixel buffers after quantization and then delegates final JPEG encoding to the libjpeg library. This approach enables:

1. **Direct quality observation** — The reconstructed image reflects actual compression losses immediately
2. **Content-aware compression** — Saliency-based quality scaling can be applied before final encoding
3. **CPU-GPU parity** — Deterministic comparison between implementations on identical data
4. **Standards compliance** — Output JPEG files conform to standard JPEG format via libjpeg

6.2 Block-Wise Image Decomposition

The input image is processed in non-overlapping 8×8 blocks, a unit size chosen for balance between computational efficiency and perceptual granularity. Each color channel (red, green, blue) is processed independently and in parallel.

For images whose dimensions are not multiples of 8, edge blocks are padded using edge-clamping: the last valid pixel value is replicated to fill the partial block. This approach avoids introducing artificial boundary artifacts.

6.3 Forward Discrete Cosine Transform (Forward DCT)

For each 8×8 pixel block, the forward 2D DCT is applied in matrix form:

$$F = TXT^T \tag{6.1}$$

where:

- **X** is the 8×8 pixel block, centered by subtracting 128 from each pixel
- **T** is the 8×8 DCT basis matrix (computed analytically from the DCT-II formula)
- **F** is the resulting 64 DCT coefficients organized as an 8×8 matrix

The DCT basis is constructed using the standard formula:

$$T_{u,x} = \alpha_u \cos\left(\frac{(2x+1)u\pi}{16}\right) \tag{6.2}$$

where:

- $u, x \in [0, 7]$
- $\alpha_0 = \frac{1}{\sqrt{8}}$
- $\alpha_u = \frac{\sqrt{2}}{8}$ for $u > 0$

The DCT transformation concentrates image energy into the low-frequency components (top-left corner of the coefficient matrix), while high-frequency components (bottom-right) capture fine detail and noise. This energy concentration property makes the transform ideal for lossy compression.

6.4 Quantization: Lossy Compression Step

After the forward DCT, each coefficient is quantized using an 8×8 quantization table:

$$Q_{u,v} = \left\lfloor \frac{F_{u,v}}{q_{u,v}} + 0.5 \right\rfloor \tag{6.3}$$

where:

- $\mathbf{q}$ is the 8×8 quantization table (values from 1 to 255)
- The rounded quotient is the quantized coefficient value
- Larger quantization values discard more information (stronger compression)

6.4.1 Quantization Table Design

This implementation uses IJG (libjpeg) baseline quantization tables (`kBaseQ_Luma` and `kBaseQ_Chroma`), which are the standard tables used by JPEG. These tables encode human visual perception: low-frequency components receive smaller quantization values (finer preservation), while high-frequency components receive larger values (aggressive compression).

6.4.2 Quality-Based Scaling

The base quantization tables are scaled by a quality factor $Q \in [1, 100]$ using the IJG scaling rule:

$$\text{scale} = \begin{cases} \frac{5000}{Q} & \text{if } Q < 50 \\ 200 - 2Q & \text{otherwise} \end{cases} \tag{6.4}$$

This produces per-quality quantization tables where:

- $Q = 100$ produces near-lossless compression (minimal quantization)
- $Q = 1$ produces maximum compression (very coarse quantization)

6.4.3 Per-Block Coefficient Scaling (Saliency-Aware)

When saliency-based quality mapping is enabled, DCT coefficients are scaled by a per-block multiplier **before** quantization:

$$F'_{u,v} = F_{u,v} \times s_b \tag{6.5}$$

where s_b is the inverse of the block's saliency importance score. This biases the quantization:

- **Important blocks** (high saliency, $s_b < 1$) receive less aggressive quantization
- **Unimportant blocks** (low saliency, $s_b > 1$) receive more aggressive quantization

This preserves visual quality in perceptually important regions (edges, textures) while allowing greater compression in uniform or low-detail areas.

6.5 Dequantization and Inverse DCT

Dequantization reverses the quantization step by multiplying each quantized coefficient by its quantization table value:

$$\hat{F}_{u,v} = Q_{u,v} \times q_{u,v} \tag{6.6}$$

The dequantized coefficients $\hat{\mathbf{F}}$ are now an approximation of the original DCT coefficients **with quantization error**. This error represents the irreversible information loss from the lossy compression step.

The **inverse 2D DCT** reconstructs pixel values from the dequantized coefficients:

$$\hat{X} = T^T \hat{F} T \tag{6.7}$$

where T^T is the transpose of the DCT basis matrix. The reconstructed block $\hat{\mathbf{X}}$ contains

8×8 pixel values, still centered (mean-shifted by -128).

6.6 Pixel Reconstruction and Clamping

After the inverse DCT, pixels are re-centered by adding 128, then clamped to the valid range $[0, 255]$ and cast to unsigned 8-bit integers:

$$\hat{x}_{i,j} = \text{clamp}\left(\text{lrint}(\hat{X}_{i,j} + 128), 0, 255\right) \tag{6.8}$$

The result is a reconstructed 8×8 block of RGB pixel values that reflects the cumulative effect of quantization losses.

6.7 RGB Image Assembly and JPEG Encoding

All reconstructed blocks for each color channel are assembled into full-size RGB image buffers. These buffers are then passed to the **libjpeg scanline encoder**, which:

1. **Applies a second quantization** — libjpeg quantizes the reconstructed pixels using its own luma/chroma tables
2. **Performs color space conversion** — Converts RGB to YCbCr (optional subsampling)
3. **Applies Huffman entropy coding** — Compresses the quantized DCT coefficients with run-length and frequency-based encoding
4. **Writes the JPEG file** — Produces a standards-compliant JPEG with appropriate headers and markers

Why this two-stage quantization? The first quantization (in this pipeline) simulates JPEG compression to show the visual quality loss. The second quantization (by libjpeg) produces the actual bitstream compression. The reconstructed pixel buffers are smoother and have less entropy than the original, allowing libjpeg to achieve significant file size reduction through entropy coding.

6.8 Quality Assessment Metrics

To evaluate compression effectiveness, two metrics are computed:

6.8.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(I_i - \hat{I}_i)^2 \tag{6.9}$$

where:

- I_i is the original pixel value
- $\hat{I}_i$ is the reconstructed pixel value
- N is the total number of pixels (`width` $\times$ `height`)

MSE measures the average pixel-level error and is sensitive to all differences between original and reconstructed images.

6.8.2 Peak Signal-to-Noise Ratio (PSNR)

$$\text{PSNR} = 10\log_{10}\left(\frac{255^2}{\text{MSE}}\right) \tag{6.10}$$

PSNR converts MSE to a logarithmic scale where higher values indicate better quality:

- **PSNR ¿ 40 dB** — Visually imperceptible differences
- **PSNR 30–40 dB** — Acceptable quality for most applications
- **PSNR ¡ 30 dB** — Visible artifacts

In compare mode, PSNR is computed between CPU and GPU reconstructed buffers to verify numerical accuracy and detect implementation differences.

6.9 CPU and GPU Implementation Parity

Both the CPU (C++) and GPU (CUDA) implementations follow the same algorithmic sequence and use identical quantization tables and mathematical constants. Key design choices ensure parity:

1. **Identical DCT matrices** — Both use the analytical DCT-II basis, computed once per invocation
2. **Same quantization tables** — Both read from `make_scaled_quant_tables()`
3. **Deterministic rounding** — Both use standard rounding (round-to-nearest-integer)
4. **Bit-level accuracy verification** — Compare mode computes PSNR to detect divergence

The CPU path serves as a reference for debugging and validation, while the GPU path leverages parallelism for performance.

6.10 Summary of the Pipeline

The complete compression pipeline is:

1. **Input decoding** — Load PNG/image using stb_image
2. **Channel decomposition** — Convert interleaved RGB to planar representation
3. **Optional saliency analysis** — Compute per-block importance scores (if enabled)
4. **Block-wise processing** — For each 8×8 block in each channel:
   (a) Apply forward 2D DCT
   (b) Scale coefficients by block importance (if saliency enabled)
   (c) Quantize coefficients using quality-scaled quantization tables
   (d) Dequantize to approximate original DCT values

(e) Apply inverse 2D DCT to reconstruct pixels

(f) Clamp and store reconstructed pixel values

5. **JPEG encoding** — Pass reconstructed RGB buffers to libjpeg for final entropy coding and JPEG file output

6. **Quality metrics** — Compute MSE and PSNR if comparison is requested

This approach makes explicit what traditional JPEG encoders hide internally, enabling detailed performance analysis and perceptually-guided compression.

7 . Experimental Setup

The performance evaluation and benchmarking of the image compression pipeline were conducted on a dedicated hardware setup. This environment was carefully chosen to measure the execution timings of both the CPU reference implementation and the GPU-accelerated CUDA kernels.

Table 7.1 details the specific hardware components, including the host processor and the discrete graphics card, alongside the software toolchain used for compiling and running the experiments. Alongside this hardware, various sample images of differing resolutions and detail levels were used to validate the compression behavior and benchmark throughput.

Component	**Specification**
GPU	NVIDIA GeForce RTX 3060 Laptop GPU (GA106, sm_86), 6 GB GDDR6
GPU CUDA Cores	3840
CPU	AMD Ryzen 7 6800H with Radeon Graphics (16 logical cores)
CUDA Toolkit	Version 12.0
Host Compiler	GCC 13.3.0
Operating System	Linux 6.17.0-22-generic

Table 7.1: Hardware and software specifications for the experimental environment

This project therefore uses experimental inputs rather than a labeled dataset. That is why the dataset section from the template is adapted here to describe the actual project workflow more accurately.

8 . Results and Discussion

The current implementation demonstrates a complete end-to-end compression pipeline on both CPU and GPU paths. The GPU version accelerates the block-based transform and reconstruction stages, while the CPU version serves as a correctness baseline. In compare mode, the project can report output file size, wall-clock timing, stage-level timings, and PSNR between CPU and GPU reconstructions.

The saliency-based quality map improves the flexibility of the compressor by allowing visually important regions to retain more detail than less important regions. This is especially useful for images with strong edges, textured objects, or regions that are more noticeable to the human eye. At the same time, the use of planar channels and block-wise processing keeps the implementation simple enough to analyze and benchmark.

From a systems perspective, one important observation is that compression performance depends not only on the kernel execution time but also on memory transfer, saliency computation, and JPEG encoding. The timing breakdown therefore gives a more complete picture than a single runtime value. The project is well suited for experimentation because it makes these trade-offs visible.

Future result tables can include measured comparisons for different quality settings, different input sizes, and runs with and without the quality map. That will make it possible to quantify the benefit of GPU acceleration and the visual trade-off introduced by content-aware compression.

```
[GPU] wrote tests/artifacts/catout-gpu.jpg in 22.625 ms
[CPU] wrote tests/artifacts/catout-cpu.jpg in 81.179 ms
path   |  time (ms) |          MSE |         PSNR
---------------------------------------------
GPU    |     22.625 |           -- |           --
CPU    |     81.179 |     0.009483 |       68.361
---------------------------------------------

Speedup (CPU/GPU): 3.59x

Stage breakdown:
GPU stage breakdown (ms): saliency=0.000 upload=0.672 compute=1.095 download=0.184 encode=18.479 total=22.625
CPU stage breakdown (ms): saliency=0.000 upload=0.000 compute=66.874 download=0.000 encode=81.059 total=81.179
```

Figure 8.1: Output example 1

```
[GPU] wrote tests/artifacts/out-gpu.jpg in 24.083 ms
[CPU] wrote tests/artifacts/out-cpu.jpg in 81.870 ms
path   |  time (ms) |          MSE |         PSNR
---------------------------------------------
GPU    |     24.083 |           -- |           --
CPU    |     81.870 |     0.016637 |       65.920
---------------------------------------------

Speedup (CPU/GPU): 3.40x

Stage breakdown:
GPU stage breakdown (ms): saliency=0.000 upload=0.714 compute=1.095 download=0.123 encode=20.076 total=24.083
CPU stage breakdown (ms): saliency=0.000 upload=0.000 compute=67.429 download=0.000 encode=81.698 total=81.870
```

Figure 8.2: Output example 2

The results shown in the above snapshots highlight the performance and quality comparison between CPU and GPU implementations of the compression pipeline. The GPU achieves a consistent speedup of over 3× due to parallel execution of 8×8 block operations. Stage-wise timing reveals that computation is significantly accelerated on the GPU, although encoding time remains comparable. The MSE values are low and PSNR values exceed 65 dB, confirming minimal quality loss. This validates that GPU acceleration improves performance without compromising reconstruction accuracy.

Conclusion

This project demonstrates a practical GPU-accelerated image compression system built for parallel and distributed computing. We've combined a CPU reference implementation with CUDA acceleration, optional content-aware quality mapping, and standard JPEG output—all designed to be modular, measurable, and easy to extend. What makes this system valuable is how it balances performance gains with code clarity, enabling developers to analyze each stage of compression in detail. Experimental observations show significant speedups on larger images, with efficient parallel processing of thousands of 8×8 blocks and minimal quality degradation, often maintaining high PSNR values. Looking forward, we plan to explore additional color models, adaptive quantization strategies, and integration with modern video codecs. These enhancements would strengthen the platform's role as both a teaching tool and a practical research resource for parallel computing, while also improving scalability and real-world applicability.

Bibliography

[1] G. K. Wallace, "The JPEG still picture compression standard," *Communications of the ACM*, 1992.

[2] D. B. Kirk and W. H. Wen-Mei, *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2016.

[3] NVIDIA, *CUDA C++ Programming Guide*.

[4] L. Itti, C. Koch, and E. Niebur, "A model of saliency-based visual attention for rapid scene analysis," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1998.